

Einführung in C - Teil 2

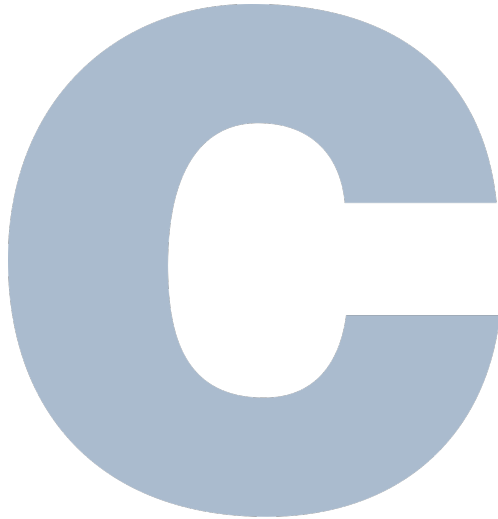


Figure 1: C-Logo

switch-case

```
switch (WERT) {  
    case WERT_1:  
        // Anweisungen  
        break;  
    case WERT_2:  
        // Anweisungen  
        break;  
    // ...  
    default:  
        // No matches;  
}
```

- ▶ switch wertet eine Variable auf Gleichheit gegen mehrere Konstanten aus.
- ▶ Entspricht der Wert in switch() einem case-Wert, werden die zugehörigen Anweisungen ausgeführt.

```
switch (WERT) {  
    case WERT_1:  
        // Anweisungen  
        break;  
    case WERT_2:  
        // Anweisungen  
        break;  
    // ...  
    default:  
        // No matches;  
}
```

- ▶ Mit break wird der switch-Block verlassen.

```
switch (WERT) {  
    case WERT_1:  
        // Anweisungen Block 1  
    case WERT_2:  
        // Anweisungen Block 2  
        break;  
    // ...  
    default:  
        // No matches;  
}
```

- Ohne break wird auch der nächste Block ausgeführt.

Beispiel

```
switch (x) {  
    case 1:  
    case 3:  
        printf("x ist ungerade\n");  
        break;  
    case 2:  
    case 4:  
        printf("x ist gerade\n");  
        break;  
    default:  
        printf("Außerhalb des Bereichs\n");  
}
```

- ▶ mit `default`: wird ein optionaler Fallback-Zweig definiert, wenn kein case passt.

Beispiel

```
switch (x) {  
    case 1:  
        printf("x ist 1\n");  
        break;  
    case 2:  
        printf("x ist 2\n");  
        break;  
    default:  
        printf("x ist weder 1 noch 2\n");  
}
```

- ▶ Der Wert in `switch()` muss ein Ganzzahl-Typ sein (z.B. `int`).
- ▶ case-Werte müssen Konstanten sein.

Operatoren

- ▶ Wir haben schon die arithmetischen, logischen und relationalen Operatoren kennengelernt.
- ▶ Ein Operator ist ein Symbol (oder eine Operator-Kombination), das einen oder mehrere Operanden verbindet bzw. transformiert, um einen neuen Wert zu berechnen.
- ▶ In C gibt es drei Arten von Operatoren: **unär**, **binär** und **ternär**.

- ▶ **unäre** Operatoren haben einen Operanden, z.B.: `!a`
- ▶ **binäre** Operatoren haben zwei Operanden, z.B.: `a + b`
- ▶ **ternäre** Operatoren haben drei Operanden, z.B.: `a ? b : c`
 - ▶ Der Ausdruck liefert `b`, wenn `a` wahr ist, sonst `c`
 - ▶ Das ist der einzige **ternäre** Operator in C.

- ▶ Die Operatoren können weiter in drei Kategorien unterteilt werden.
- ▶ Bei **Präfix**-Operatoren steht der Operator vor dem Operanden, z.B. `!a`
- ▶ Bei **Infix**-Operatoren steht der Operator zwischen den Operanden, z.B. `a + b`
- ▶ Bei **Postfix**-Operatoren steht der Operator hinter dem Operanden, z.B. `a++`

Assoziativität von Operatoren

- ▶ Operatoren können **linksassoziativ** oder **rechtsassoziativ** sein.
- ▶ **linksassoziativ** bedeutet, dass von links nach rechts ausgewertet wird.
- ▶ **rechtsassoziativ** bedeutet entsprechend von rechts nach links.
- ▶ Die logischen, relationalen und arithmetischen Operatoren sind **linksassoziativ**.
- ▶ Bei dem Ausdruck $a + b - c$ wird zuerst $a + b$ gerechnet, dann c abgezogen.
- ▶ **rechtsassoziativ** sind $=$, $+=$, $-=$, $*=$, $/=$ und $\%=$

Die Increment-, Decrement- und Compound-Assignment Operatoren

Operator	Beispiel	Bedeutung
++ (postfix)	a++	Gibt a zurück und zählt dann a um 1 hoch
++ (präfix)	++a	Erhöht a um 1 und liefert dann a zurück

Die Increment-, Decrement- und Compound-Assignment Operatoren (Fs.)

Operator	Beispiel	Bedeutung
-- (postfix)	a--	Gib a zurück und zählt a um 1 runter
-- (präfix)	--a	Zähle a um 1 runter und liefere dann a zurück

Die Increment-, Decrement- und Compound-Assignment Operatoren (Fs.)

Operator	Beispiel	Bedeutung
+=	a += 2	Erhöhe a um 2 und gib dann das Ergebnis zurück
--	a -= 2	Zähle a um 2 runter und gib dann das Ergebnis zurück

Die Increment-, Decrement- und Compound-Assignment Operatoren (Fs.)

Operator	Beispiel	Bedeutung
<code>*=</code>	<code>a *= 2</code>	Setze <code>a = a * 2</code> und gib dann das Ergebnis zurück
<code>/=</code>	<code>a /= 2</code>	Setze <code>a = a / 2</code> und gib dann das Ergebnis zurück
<code>%=</code>	<code>a %= 2</code>	Setze <code>a = a % 2</code> und gib dann das Ergebnis zurück

Beispiel

```
int a = 2, b;
```

```
b = a++; // b = 2
```

```
        // a ist jetzt 3
```

```
b = ++a; // a = 4, b = 4
```

```
a *= 4;  // a = 16
```

```
a--;    // a = 15
```

```
a /= 2; // a = 7 (Ganzzahldivision)
```

```
a %= 2; // a = 1 (Rest 1 bei Division durch 2)
```

Schleifen

- ▶ Wiederholung eines Codeblocks solange eine Bedingung erfüllt ist.
- ▶ In C gibt es drei Schleifenarten: `for`, `while` und `do-while`.
- ▶ Ein einzelner Durchlauf des Schleifenkörpers, bei dem die Anweisungen der Schleife einmal ausgeführt werden, nennt man **Schleifeniteration**.

while-Schleifen

```
while (BEDINGUNG) {  
    // Code, der wiederholt werden soll  
}
```

- ▶ Ein while-Schleife wiederholt den Code im Schleifenkörper solange die Bedingung im Schleifenkopf wahr ist.
- ▶ while-Schleifen nennt man daher kopfgesteuert.
- ▶ Die Bedingung im Schleifenkopf ist ein Boolescher Ausdruck, so wie bei if.
- ▶ Enthält der Schleifenkörper mehr als eine Anweisung, wird der Code in { ... } eingeschlossen.

Beispiel

```
int counter = 0;

while (counter < 5) {
    printf("%d\n", counter);
    counter++;
}
```

Ausgabe

0
1
2
3
4

- ▶ Es wird erst geprüft, ob die Bedingung wahr ist. $0 < 5$ ist offensichtlich wahr.
- ▶ Im Schleifenkörper wird dann der Wert des Zählers ausgegeben.
- ▶ Anschließend wird `counter` mit dem *Increment*-Operator um 1 hochgezählt.
- ▶ Dann wird wieder zum Schleifenkopf gesprungen
- ▶ etc.

Beispiel

Wie oft wird diese Schleife ausgeführt?

```
int counter = 5;

while (counter < 5) {
    printf("%d\n", counter);
    counter++;
}
```

Beispiel

Was passiert hier?

```
int counter = 0;

while (counter < 5) {
    printf("%d\n", counter);
}
```

do-while-Schleifen

```
do {  
    // Code, der wiederholt werden soll  
} while (BEDINGUNG)
```

- ▶ Im Gegensatz zur while-Schleife ist die do-while-Schleife fußgesteuert.
- ▶ D.h., es wird mindestens einmal der Schleifenkörper ausgeführt und dann geprüft.
- ▶ Es gilt sonst das gleiche wie bei der while-Schleife.
- ▶ do-while-Schleifen kommen relativ selten vor.

Beispiel

```
int counter = 0;

do {
    printf("%d\n", counter);
    counter++;
} while (counter < 5);
```

Ausgabe

0
1
2
3
4

Beispiel

Was passiert hier?

```
int counter = 0;
do {
    printf("%d\n", counter);
    counter++;
} while (0);
```

for-Schleifen

```
for (INITIALISIERUNG; BEDINGUNG; UPDATE) {  
    // Code, der wiederholt wird  
}
```

- ▶ Die for-Schleife wiederholt den Code im Schleifenkörper, solange die Bedingung wahr ist.
- ▶ Die Bedingung ist auch hier wieder ein Boolescher Ausdruck.
- ▶ Zuerst wird eine Initialisierung ausgeführt.
- ▶ Anschließend beginnt die eigentliche Schleife:
 1. Bedingung prüfen
 2. Schleifenkörper ausführen, wenn Bedingung wahr ist, sonst Ende.
 3. Update von z.B. einer Zählvariablen
 4. Sprung zu 1.

Beispiel

```
for (int i = 0; i < 5; i++) {  
    printf("%d\n", i);  
}
```

Ausgabe

0
1
2
3
4

Endlosschleifen

- ▶ Endlosschleifen können wir mit allen drei Schleifenformen erzeugen.
- ▶ Dazu muss die Bedingung, die der Schleifenkopf/fuß prüft, immer wahr sein.

Endlos-while- und do-while-Schleifen

```
while (1) { // Ist immer wahr  
    //...  
}
```

```
do {  
    // ...  
} while (1); // Ist immer wahr
```

Endlos-for-Schleifen

Bei for brauchen wir lediglich die Bedingung wegzulassen.

```
for (;;) {  
    // ...  
}
```

Schleifen vorzeitig verlassen mit break

- ▶ Alle Schleifen lassen sich mit break verlassen

Beispiel

```
// Gib die Zahlen 0 bis 4 aus.  
for (int i = 0; i < 100; i++) {  
    if (i >= 5)  
        break;  
    printf("%d\n", i);  
}
```

Schleifen-Iteration unterbrechen mit `continue`

- ▶ Die `continue`-Anweisung bewirkt, dass eine Schleife an dieser Stelle unterbrochen und die nächste Iteration ausgeführt wird.

Beispiel

```
for (int i = 0; i < 7; i++) {  
    if (i % 2 != 0)  
        continue;  
    printf("%d\n", i);  
}
```

Wenn i ungerade ist, gehe zum Anfang der Schleife für das nächste i .

Ausgabe

0
2
4
6

Bemerkung

- ▶ Bei einer `do-while`-Schleife bewirkt `continue` einen Sprung zum Schleifenfuß.
- ▶ Bei `for`- und `while`-schleifen wird zum Schleifenkopf gesprungen.

Verschachtelte Schleifen

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 3; j++) {  
        // Anweisungen  
    }  
}
```

- ▶ Verschachtelte Schleifen sind Schleifen, die in den Rumpf einer anderen Schleife eingebettet sind.
- ▶ Eine innere Schleife führt ihre vollständigen Iterationen für jede Iteration der äußeren Schleife aus.
- ▶ Die Laufvariablen der inneren und äußeren Schleife müssen unabhängig sein.
- ▶ `break/continue` gelten jeweils für die jeweilige Schleife, in der sie stehen.

Beispiel

```
for (int i = 0; i < 5; i++) {  
    for (int j = 1; j <= 5; j++) {  
        printf("%d ", j);  
    }  
    printf("\n");  
}
```

Ausgabe

```
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5
```

Arrays

- ▶ Ein Array ist eine zusammenhängende Sammlung von Elementen desselben Typs im Speicher.
- ▶ Elemente werden über einen 0-basierten Index angesprochen ($a[0]$ bis $a[n - 1]$).
- ▶ Arrays haben eine feste Länge.
- ▶ Der Arrayname verweist auf die Startadresse des Arrays im Speicher.
- ▶ Arrays werden verwendet für Tabellen/Listen fester Länge, Verarbeitung von Sequenzen

Standardwerte eines Arrays

```
int array[10];
```

- ▶ Legt man ein Array an, ohne es explizit zu initialisieren, ist der Inhalt undefiniert (zufällige Werte).
- ▶ Global und statisch definierte Arrays werden mit Nullen initialisiert.

Explizite Initialisierung eines Arrays

- ▶ Explizite Initialisierung bedeutet, dass die Anfangswerte eines Arrays direkt bei der Deklaration festgelegt werden.
- ▶ Das geschieht typischerweise mit geschweiften Klammern, z. B.
`int a[3] = {1, 2, 3};`
- ▶ Die Anzahl der Initialisierungswerte kann kleiner oder gleich der Arraygröße sein; fehlende Elemente werden automatisch mit 0 gefüllt.
- ▶ Wird die Größe nicht angegeben, leitet der Compiler sie aus der Anzahl der Elemente ab.

Beispiel

// Lege ein int-Array mit 5 Primzahlen an.

```
int numbers[] = { 2, 3, 5, 7, 11 };
```

```
printf("%d\n", numbers[3]); // Gibt 7 aus.
```

```
printf("%d\n", numbers[0]); // Gibt 2 aus.
```

```
printf("%d\n", numbers[1]); // Gibt 3 aus.
```

Beispiel

```
int numbers[] = { 2, 3, 5, 7, 11 };  
// Gib alle Zahlen bis auf die erste aus  
for (int i = 1; i < 5; i++) {  
    printf("%d\n", numbers[i]);  
}
```

Ausgabe

3
5
7
11

Werte eines Arrays ändern

Mithilfe des Zuweisungsoperators = können wir Elemente eines Arrays ändern:

```
int array[4] = { 2, 3, 4, 7 };  
array[2] = 5; // Ändere 4 auf 5
```

Vorsicht bei der Indizierung

```
int array[4] = {2, 3, 5, 7};
```

```
array[4] = 11; // Element 4 existiert nicht!
```

```
printf("%d\n", array[4]); // Nein!
```

```
printf("%d\n", array[-1]); // Stopp!
```

- ▶ Greift man lesend oder schreibend auf Elemente außerhalb der Array-Grenzen zu, ist das Verhalten undefiniert.
- ▶ Oft ist die Folge ein Absturz des Programms (Segmentation fault).
- ▶ Das Schreiben über die Grenzen hinweg nennt man einen Buffer Overflow.
- ▶ C ist berüchtigt für Buffer Overflow-Bugs.

Größe eines Arrays bestimmen

- ▶ Mit dem sizeof-Operator können wir die Größe eines Arrays bestimmen.
- ▶ Die Anzahl der Elemente ist `sizeof(array) / sizeof(typ)`

```
int numbers[] = { 2, 3, 5, 7, 11 };  
// size = (20 = 5 * 4)  
size_t size = sizeof(numbers);  
// n = 20 / 4 = 5  
size_t n = sizeof(numbers) / sizeof(int);  
// Oder sizeof(numbers) / sizeof(numbers[0])
```

Arrays vergleichen

- ▶ Arrays können nur elementweise verglichen werden.
- ▶ Ein Ausdruck wie `array1 == array2` vergleicht die Speicher-Adressen der beiden Arrays, **nicht deren Inhalt**.

Beispiel

```
int a[] = { 1, 2, 3, 4 };
int b[] = { 1, 2, 3, 5 };
int equal = 1;

if (sizeof(a) != sizeof(b))
    equal = 0;
for (int i = 0; equal && i < sizeof(a); i++) {
    if (a[i] != b[i])
        equal = 0;
}
```


Mehrdimensionale Arrays

```
int a[2][3];
```

- ▶ Ein mehrdimensionales Array ist ein Array, dessen Elemente selbst wieder Arrays sind.
- ▶ Sie eignen sich zur Darstellung von Daten in Tabellen, Matrizen oder Gittern.

Beispiel

// Explizite Initialisierung

```
int a[2][3] = { {1,2,3},  
               {4,5,6} };
```

// Element in Zeile 2, Reihe 3

```
printf("%d\n", a[1][2]); // 6
```

// Element in Zeile 1, Reihe 2

```
printf("%d\n", a[0][1]); // 2
```

Strings

- ▶ C-Strings sind nullterminierte Zeichenketten.
- ▶ Ein C-String besteht aus einer Folge von char-Zeichen, die mit dem Nullzeichen `\0` endet.
- ▶ Ein C-String kann in einem Array, im dynamisch reservierten Speicher (Heap) oder in einem String-Literal liegen.
- ▶ C besitzt keinen eigenen String-Datentyp.
- ▶ Viele Funktionen zum Bearbeiten von C-Strings stellt die Standardbibliothek über `<string.h>` bereit.

Speicher-Layout eines Strings



Figure 2: C-String

String-Literale

- ▶ String-Literale sind Zeichenketten eingeschlossen in `"`, wie etwa `"hello"`.
- ▶ Sie liegen in einem statischen, unveränderlichen Speicherbereich und existieren während der gesamten Programmlaufzeit.
- ▶ Schreiben wir `char s[] = "hello"`, kopiert der Compiler das String-Literal in das Array.
- ▶ Die Zeichen im Array selbst sind änderbar, das String-Literal nicht.

Beispiel

```
char str1[] = "hello, world!";  
// Umständlicher, aber gleichbedeutend  
char str2[] = {'h', 'e', 'l', 'l', 'o', '\\0' };  
printf("%s\\n%s\\n", str1, str2);
```

Ausgabe

```
Hello, world!  
hello
```

Zugriff auf Zeichen eines Strings

- ▶ Die Zeichen eines C-Strings liegen zusammenhängend im Speicher.
- ▶ Jedes Zeichen kann über seinen Index angesprochen werden, beginnend bei 0.
- ▶ Der Ausdruck `string[i]` greift auf das Zeichen an der Position `i` zu.

Beispiel

```
char str[] = "hello";  
// hello -> Hello  
str[0] = 'H';  
// Länge = 5 Zeichen  
for (int i = 0; i < 5; i++)  
    printf("%c ", str[i]);  
printf("\n");
```

Ausgabe

H e l l o

Länge von String-Literalen und String-Arrays

- ▶ Die Länge eines String-Literals oder eines mit String-Literal definierten Arrays inkl. Nullbyte können wir mit `sizeof()` bestimmen.
- ▶ Allgemein entspricht die Länge eines Arrays nicht der Länge des darin gespeicherten Strings (mit Nullbyte).
- ▶ Hierfür stellt die Standardbibliothek eine Lösung bereit.

Beispiel

```
char str[] = "hello";  
char buf[] = "hello\0\0\0\0";  
size_t l1 = sizeof("hello"); // 6  
size_t l2 = sizeof(str);      // 6  
size_t l3 = sizeof(buf);      // 10
```

Vergleich und Zuweisung von Strings

- ▶ In C können wir weder die Vergleichsoperatoren noch den Zuweisungsoperator mit Strings nutzen.
- ▶ Dafür gibt es spezielle Funktionen aus der Standardbibliothek.
- ▶ Schreiben wir `str1 == str2` vergleichen wir die Speicher-Adressen der beiden Strings, **nicht aber deren Inhalt**.

```
char s1[] = "foo";  
char s2[] = "bar";  
char s3[4];
```

```
s1 == s2; // Nein  
s3 = s2;  // Nein
```

Strings in mehrdimensionalen Arrays Speichern

- ▶ Um mehrere Strings zu speichern, verwendet man ein zweidimensionales char-Array, z. B. `char namen[10][50];`, also Platz für 10 Strings à max. 49 Zeichen + `\0`.
- ▶ Die Strings liegen zeilenweise im Speicher; jede Zeile ist ein separater String.

Beispiel

```
char strings[][12] = {  
    "Hello",  
    "world!",  
    "How",  
    "are",  
    "you?"  
};  
for (int i = 0; i < sizeof(strings) / 12; i++)  
    printf("%s ", strings[i]);  
printf("\n");
```

Ausgabe

Hello world! How are you?

Strings mit `scanf()` einlesen

- ▶ Strings lassen sich mit dem Formatstring-Platzhalter `%s` einlesen.
- ▶ Dabei ist es wichtig, die Anzahl der einzulesenden Zeichen zu begrenzen.
- ▶ String mit Leerzeichen können nicht eingegeben werden, da `scanf()` diese zur Feldtrennung benutzt.

Beispiel

```
char buf[20];  
// Das letzte Byte ist das Nullbyte.  
printf("Eingabe: ");  
scanf("%19s", buf);  
printf("Ausgabe: %s\n", buf);
```

Ausgabe

Eingabe: Hello, world!

Ausgabe: Hello,

Besser: `fgets()`

```
char *fgets(char buf[], int size, FILE *stream);
```

- ▶ `fgets()` liest max. `size - 1` Bytes vom `FILE`-Stream `stream` und speichert das Ergebnis inklusive Nullbyte in `buf`.
- ▶ `fgets()` liest dabei bis zum ersten Newline `\n` oder bis das Ende der Datei (EOF) erreicht ist.
- ▶ Bei einem Fehler gibt `fgets()` `NULL` zurück.
- ▶ Für die Eingabe von der Tastatur benutzen wir den Stream `stdin`.

Beispiel

```
char buf[20];  
printf("Eingabe: ");  
fgets(buf, sizeof(buf), stdin);  
printf("Ausgabe: [%s]", buf);
```

Ausgabe

Das Newline-Zeichen (Return/Enter-Taste) wird mit gespeichert:

```
Eingabe: Hello, world!  
Ausgabe: [Hello, world!  
]
```

<string.h>

- ▶ `string.h` stellt Funktionen zum Kopieren, Vergleichen, Suchen und Verketteten von C-Strings bereit.
- ▶ Die meisten String-Funktionen erwarten nullterminierte Zeichenketten als Eingabe.
- ▶ Beim Einsatz der Funktionen muss darauf geachtet werden, dass ausreichend Speicher vorhanden ist, um Pufferüberläufe und undefiniertes Verhalten zu vermeiden.

strlen()

Diese Funktion liefert die Länge des übergebenen Strings bis zum terminierenden Nullbyte zurück.

```
size_t strlen(const char *str);
```

Beispiel

```
char str[] = "hello\0\0\0\0";  
printf("sizeof(str): %lu\n", sizeof(str));  
printf("Länge (str): %lu\n", strlen(str));  
printf("Länge (hello): %lu\n", strlen("hello"));
```

Ausgabe

```
sizeof(str): 10  
Länge (str): 5  
Länge (hello): 5
```

strcmp()

- ▶ strcmp() vergleicht zwei nullterminierte C-Strings Zeichen für Zeichen.
- ▶ Der Vergleich erfolgt lexikografisch anhand der Zeichenwerte.
- ▶ Die Funktion vergleicht die Strings bis zum ersten unterschiedlichen Zeichen oder bis zum Nullzeichen \0.
- ▶ Rückgabewert:
 - ▶ 0, wenn beide Strings gleich sind.
 - ▶ Kleiner als 0, wenn der erste String lexikografisch kleiner ist als der zweite.
 - ▶ Größer als 0, wenn der erste String lexikografisch größer ist als der zweite.

```
int strcmp(const char *s1, const char *s2);
```

Beispiel

```
const char s1[] = "hello";  
printf("%d\n", strcmp(s1, "hello"));  
printf("%d\n", strcmp(s1, "hell"));  
printf("%d\n", strcmp(s1, "hellooo"));
```

Ausgabe

0

1

-1

strcpy()

- ▶ strcpy() kopiert einen C-String von einer Quelle (src) in ein Ziel (dst).
- ▶ Dabei werden alle Zeichen einschließlich des Nullzeichens '\0' kopiert.
- ▶ Der Zielspeicher muss groß genug sein, um den gesamten String aufzunehmen.
- ▶ Die Funktion führt keine Überprüfung der Puffergröße durch.
- ▶ Ist der Zielpuffer zu klein, kann es zu Pufferüberläufen und undefiniertem Verhalten kommen.
- ▶ Rückgabewert ist ein Zeiger auf den Zielstring (dst).

```
char *strcpy(char *dst, const char *src);
```

Beispiel

```
char ziel[12];  
const char quelle[] = "hello";  
  
strcpy(ziel, quelle);  
  
printf("%s = %s\n", ziel, quelle);
```

Ausgabe

```
hello = hello
```


Beispiel

```
char ziel[4];
```

```
// Crash
```

```
strcpy(ziel, "ein viel zu langer String");
```

Ausgabe

```
segmentation fault (core dumped)
```

Besser: strncpy()

- ▶ strncpy() kopiert höchstens n Zeichen eines C-Strings von einer Quelle (src) in ein Ziel (dst).
- ▶ Werden weniger als n Zeichen kopiert, wird der verbleibende Bereich im Ziel mit Nullzeichen (\0) aufgefüllt.
- ▶ Ist die Quelle mindestens n Zeichen lang, wird kein abschließendes Nullzeichen angefügt.
- ▶ Der Zielspeicher muss mindestens n Zeichen groß sein.
- ▶ Rückgabewert ist ein Zeiger auf den Zielstring (dst).

```
char *strncpy(char *dst, const char *src, size_t n);
```

Beispiel

```
char ziel[8];  
char quelle[] = "hello";  
  
// Kopiere 5 Zeichen von "hello" nach  
// ziel and hänge 3 \0 an.  
strncpy(ziel, quelle, sizeof(ziel));  
// ziel = hello\0\0\0
```

Beispiel

```
char ziel[4];  
char quelle[] = "hello";  
  
// Kopiere (4 - 1 = 3) Bytes nach ziel.  
strncpy(ziel, quelle, sizeof(ziel) - 1);  
// Hänge das Nullbyte an, um den String  
// zu terminieren  
ziel[3] = '\\0'; // = hel\\0
```

Newline-Zeichen entfernen mit `strcspn()`

- ▶ Wenn wir Strings mit `fgets()` einlesen, lesen wir das Newline-Zeichen `\n` mit.
- ▶ Um mögliche `\n` zu entfernen, können wir `strcspn()` benutzen.

Beispiel

```
#include <string.h>
// ...

char s[256];

if (fgets(s, sizeof s, stdin)) {
    size_t n = strcspn(s, "\n");
    s[n] = '\0';
}
```

Ende

```
printf("Danke für Eure Aufmerksamkeit!\n");  
return (0);
```

